

RAG-2: Why RAG Breaks & What Comes Next

Complete Beginner-Friendly Notes

Topics covered: Context Rot, Distractors Problem, ChatGPT's Memory Architecture, Advanced RAG Techniques, HyDE, PageIndex (Vectorless RAG), The RAG Evolution Spectrum

The Big Picture — What This Class Is About

In Class 7a, you built a RAG pipeline that worked on demo data. This class is about why that pipeline **breaks on real data** and what the industry is doing about it.

The key insight: giving an LLM more context doesn't always make it smarter. Sometimes it makes it dumber. Understanding why — and how to fix it — is what separates demo RAG from production RAG.

Part 1: Context Rot — The Hidden Killer

The Promise vs The Reality

The promise everyone believed: "Modern LLMs have 1 million+ token context windows! Just stuff all your documents in and the model will find the answer. RAG is solved!"

The reality (proven by Chroma Research testing 18 LLMs in 2025): The 10,000th token is NOT treated as reliably as the 100th. Performance degrades unpredictably as input length grows — even on simple tasks. More tokens often equals worse answers.

Why Does This Happen? The Attention Budget

Remember from Class 5: self-attention computes a score between every token and every other token. This attention is a **finite resource** — like a budget.

When you have 100 tokens, each token can "pay attention" to 99 others. The attention is concentrated. When you have 100,000 tokens, each token must spread its attention across 99,999 others. The attention gets stretched thin and diluted.

Think of it this way: if someone asks you to read a 2-page essay and find a specific fact, you'll nail it. If someone asks you to read a 500-page novel and find that same fact, you'll probably miss it — not because you can't read, but because your attention gets overwhelmed by all the other text.

The Context Rot Experiment (Notebook Part 1)

The notebook proves this with a clever experiment. It creates a "multi-hop" reasoning task:

- **Fact A:** "Dr. Elena Vasquez leads the AURORA-7 initiative."
- **Fact B:** "Dr. Vasquez confirmed the launch window opens on March 15, 2024."
- **Question:** "What is the launch date for AURORA-7?"

To answer correctly, the model must find BOTH facts and connect them (Vasquez → AURORA-7, Vasquez → March 15, therefore AURORA-7 → March 15). This is harder than just finding one fact.

The experiment buries these two facts inside increasing amounts of filler text (irrelevant paragraphs about space topics) and measures accuracy:

python

```
for n_filler in [0, 10, 30, 50, 100, 200]:  
    haystack = build_multihop_haystack(n_filler)  
    # Add filler paragraphs between the two facts  
    # Ask the model the question  
    # Check if it gets "March 15, 2024"
```

Result: With 0 filler paragraphs, the model gets it right every time. As filler increases, accuracy drops dramatically. The model starts giving wrong dates, wrong project names, or saying "I don't know."

The Distractors Problem — Even Worse

It gets worse when the filler text is **relevant but wrong** (distractors). Instead of random filler, imagine 4 paragraphs about other space projects with different

dates:

- "Project HELIOS launches April 8, 2024"
- "The TITAN-3 mission begins June 22, 2024"
- "NEBULA initiative targets September 1, 2024"

Now the model sees FIVE dates and must pick the right one. Short context: over 90% accuracy. At 32K tokens: accuracy tanks. The model follows the narrative flow and gets confused by the similar-but-wrong information.

Key insight: Well-written, coherent documents are actually HARDER for RAG than random text. The model follows the document's narrative arc instead of finding the specific needle.

How ChatGPT Actually Manages Memory

The Common Assumption

Most developers think: "ChatGPT remembers past conversations using RAG — it embeds your chat history into vectors and searches it when you reference something from last week."

The Shocking Reality

Reverse-engineering reveals: **No vector databases. No RAG. No embeddings search.** ChatGPT uses a completely different, much more surgical approach with 4 layers:

Layer 1 — Session Metadata: Your device, location, subscription tier, usage patterns. Injected once when you start chatting. Very cheap, very small.

Layer 2 — Explicit Facts: Long-term preferences stored as plain text strings. Things like "User prefers Python over JavaScript" or "User is a college student." Not embeddings — just text, curated by the model.

Layer 3 — Conversation Summaries: Approximately 15 lightweight summaries of recent chats. Only user messages are included, not the assistant's responses. These are short digests, not full transcripts.

Layer 4 — Current Session: The full sliding window of the active conversation.

This is **context engineering** — being extremely surgical about what goes into the prompt. Every token added is a bet against accuracy. ChatGPT doesn't retrieve everything that might be relevant; it includes only what's almost certainly needed.

Part 2: Advanced RAG — Making Retrieval Smarter

The notebook builds an advanced pipeline with 6 ACME Corp documents covering authentication, deployment, incident response, remote work policy, API rate limiting, and database migration.

2.1 Semantic Search (Recap)

Uses `all-MiniLM-L6-v2` to encode documents and queries into vectors, then finds the most similar ones by cosine similarity. Same as Class 7a.

python

```
embedder = SentenceTransformer("all-MiniLM-L6-v2")
doc_embeddings = embedder.encode(doc_texts, normalize_embeddings=True)

def semantic_search(query, top_k=3):
    q_emb = embedder.encode([query], normalize_embeddings=True)
    scores = np.dot(doc_embeddings, q_emb.T).flatten()
    # Return top-k by score
```

2.2 BM25 Keyword Search

Classic keyword matching — counts how often query words appear in each document, weighted by rarity. No AI, no embeddings, just word counting.

python

```
from rank_bm25 import BM25Okapi
bm25 = BM25Okapi(tokenized_docs)

def bm25_search(query, top_k=3):
    scores = bm25.get_scores(query.lower().split())
    # Return top-k by score
```

2.3 Hybrid Search (RRF Fusion)

Combines semantic and BM25 rankings using Reciprocal Rank Fusion. If a document ranks high in BOTH lists, it gets a much higher combined score.

python

```
def hybrid_search(query, top_k=3, k=60):
    # Get rankings from both methods
    # For each doc: RRF_score = 1/(k+rank_semantic) + 1/(k+rank_bm25)
    # Sort by combined RRF score
```

2.4 LLM-Based Reranking

Instead of a cross-encoder model, this notebook uses the LLM itself as a reranker. It shows the LLM the query and all candidate documents, and asks it to rank them by relevance.

python

```
def rerank(query, candidates, top_k=3):
    prompt = f'Given the query: "{query}"\n\nRank these documents by relevance.'
    # LLM returns a JSON array of indices in order of relevance
```

This is slower but more accurate than cosine similarity because the model sees the query and document **together** (like a cross-encoder).

2.5 HyDE — Hypothetical Document Embeddings

The clever trick from the slides: instead of searching with the question, generate a fake answer first, then search with that.

python

```
def hyde_search(query, top_k=3):
    # Step 1: LLM generates a hypothetical answer
    hypo_answer = chat("Write a paragraph that would answer: " + query)

    # Step 2: Embed the fake answer (not the question!)
    hypo_emb = embedder.encode([hypo_answer])

    # Step 3: Search with the fake answer's embedding
    scores = np.dot(doc_embeddings, hypo_emb.T).flatten()
```

Why this works: A question like "How does the auth system work?" is short and question-shaped. The documents are long and answer-shaped. There's a semantic gap between them. But a fake answer like "The authentication system uses OAuth 2.0 with PKCE flow..." looks much more like the actual documents, so it matches better.

Searching "answer-to-answer" is more accurate than "question-to-answer."

2.6 The Full Advanced Pipeline

python

```
def advanced_rag(query, use_hyde=True, use_rerank=True):  
    # Step 1: HyDE query transformation (optional)  
    # Step 2: Hybrid search (semantic + BM25 via RRF)  
    # Step 3: Rerank with LLM (optional)  
    # Step 4: Generate answer from top-3 chunks
```

The notebook tests this with queries like:

- "How do we handle authentication token refresh?" (straightforward)
 - "JIRA-2847" (exact string — needs BM25)
 - "What should I do if I suspect a data breach?" (cross-document)
-

Part 3: PageIndex — Vectorless, Reasoning-Based RAG

The Radical Idea: Similarity ≠ Relevance

All the RAG techniques so far rely on one assumption: that finding text **similar** to the query will find the **relevant** answer. But similarity and relevance are not the same thing.

If you ask "What was the company's revenue growth rate?", the vector search might return a paragraph about "growth strategies" (semantically similar!) instead of the actual revenue numbers (relevant but different words).

How PageIndex Works

PageIndex throws away embeddings entirely. Instead:

1. **Build a tree index** from the document — preserving the original structure (chapters, sections, subsections, paragraphs). No chunking needed.
2. **The LLM reads the tree** (just titles and summaries of each section, not the full content) and **reasons** about which section would contain the answer.
3. **Navigate down** — the LLM picks the most promising section, reads it, then either answers or goes deeper.

This is like how a human uses a textbook: you look at the table of contents, pick the right chapter, scan the section headers, and read only the relevant paragraphs. You don't read every page sequentially.

The Code

python

```
from pageindex import PageIndexClient
pi_client = PageIndexClient(api_key=PAGEINDEX_API_KEY)

# Submit a PDF – PageIndex builds the tree automatically
result = pi_client.submit_document("./sample_paper.pdf")
doc_id = result["doc_id"]

# View the tree structure
tree = pi_client.get_tree(doc_id)
# Shows: Chapter 1 → Section 1.1 → Subsection 1.1.1 etc.

# Query – LLM navigates the tree to find the answer
answer = pi_client.query(doc_id, "What training methodology did they use?")
```

Peeking Behind the Curtain

The notebook also shows how to manually give the tree to a free LLM and watch it reason:

python

```
tree_text = tree_to_text(tree) # Convert tree to readable format

prompt = f"""Here is a document's structure:
{tree_text}

Question: {query}

Which section(s) should I read to find the answer?
Explain your reasoning."""
```

The model might respond: "The question asks about training methodology. Section 3 'Methods' subsection 3.2 'Training Details' (pages 7-9) is most likely to contain this information. I'd also check Section 4 'Experiments' for implementation details."

This is **reasoning-based retrieval** — the model actively thinks about where to look, rather than passively receiving whatever the vector search returns.

The Numbers: Vector RAG vs PageIndex

On the FinanceBench benchmark (questions about financial documents):

Approach	Accuracy
Naive Vector RAG	54%
PageIndex	98.7%

The difference is enormous because financial documents are highly structured (tables, sections, appendices) and chunking destroys that structure. PageIndex preserves it.

Feature	Vector RAG	PageIndex
Retrieval method	Similarity (embeddings)	Reasoning (tree navigation)
Document structure	Destroyed by chunking	Preserved (pages/sections)
Navigation	Passive (top-K)	Active (LLM walks the tree)
Multi-hop questions	Hard / unreliable	Natural / native

The RAG Evolution Spectrum

The slides lay out 4 phases of how retrieval is evolving:

Phase 1 — Naive RAG (Passive Retrieval): Embed, search, stuff. The model passively receives whatever the retriever finds. Simple but fragile.

Phase 2 — Advanced RAG (Smarter Retrieval): Hybrid search, reranking, HyDE, contextual retrieval. Still similarity-based, but with multiple layers of improvement. This is where most production systems are today.

Phase 3 — PageIndex (Reasoning Retrieval): Vectorless, tree-based navigation. The model reasons about document structure to find answers. Preserves structure that chunking destroys.

Phase 4 — RLM (Active Exploration): Recursive Language Models. The model writes code to explore context as an environment — treating documents like a codebase it can search, filter, and navigate programmatically.

The trend: Giving the model more and more control over its own context. From "here's what the retriever found, answer based on this" to "here's the document, figure out what you need."

Making RAG Work: The Three Pillars

The slides organize all advanced techniques into three categories:

1. Better Retrieval

- **Hybrid search:** BM25 + semantic, merged with RRF
- **Reranking:** Cross-encoder or LLM rescores candidates
- **Query transformation:** HyDE, multi-query, step-back prompting

2. Better Chunking

- **Semantic chunking:** Split by meaning, not character count
- **Parent-child chunks:** Embed small snippets for matching, retrieve the larger parent chunk for context
- **Metadata filtering:** Narrow search by date, source, or type BEFORE vector search

3. Better Generation

- **Citation & attribution:** Force the model to link every claim to a source chunk
- **Self-verification:** Have the model check if its answer is actually supported by the context
- **Context engineering:** Be surgical about what the model sees — every token added is a bet against accuracy

The golden rule: "RAG isn't dead — but naive RAG is."

Master Summary Table

Concept	What It Is	Why It Matters
Context rot	LLM accuracy degrades as context length grows	"Just add more context" is a myth — less is often more
Distractors	Similar-but-wrong chunks confuse the model	Top-K retrieval doesn't guarantee the right K
Attention budget	Self-attention is finite; more tokens = thinner attention	Explains WHY context rot happens mechanically

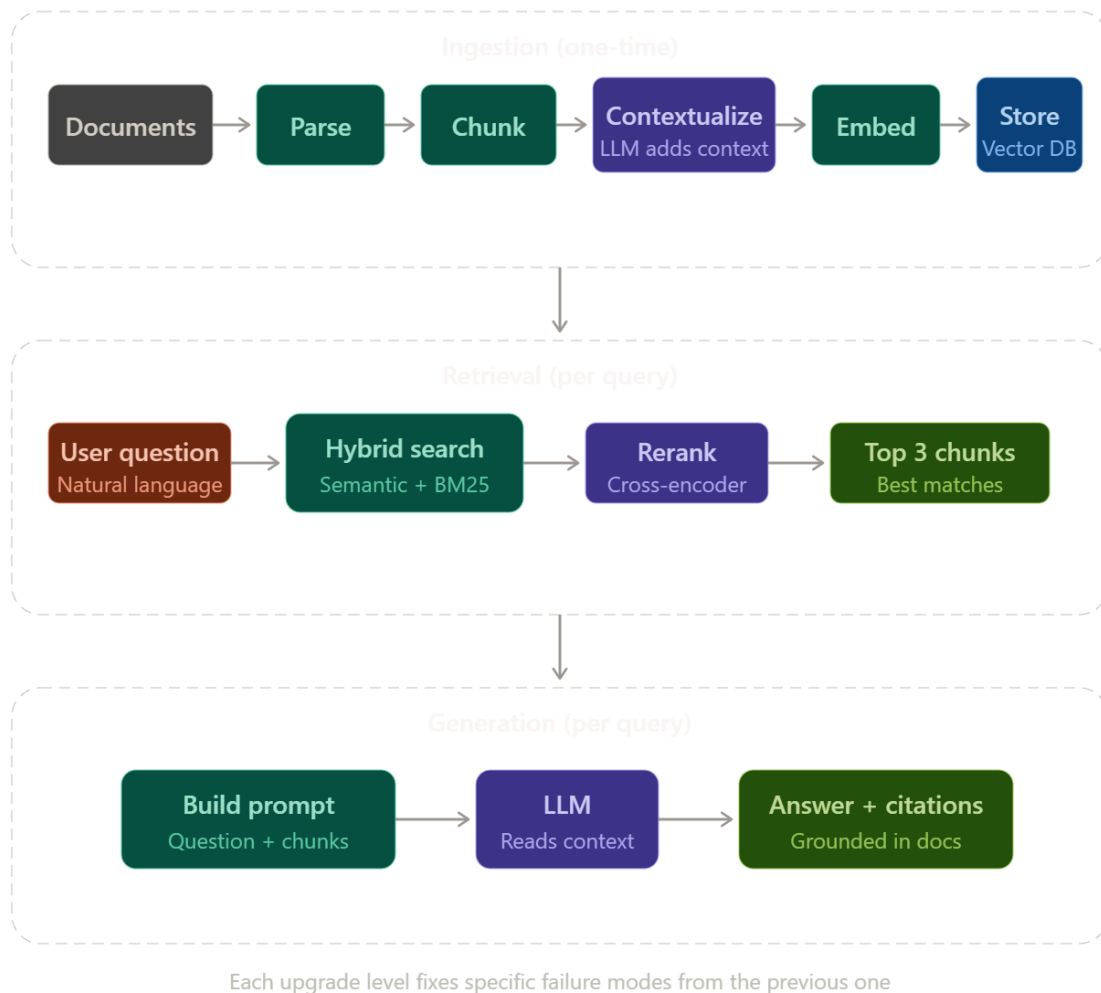
Concept	What It Is	Why It Matters
ChatGPT memory	4 layers of surgical context, no vector DB	Production systems use context engineering, not brute-force RAG
HyDE	Generate fake answer, search with that instead of question	"Answer-to-answer" matching beats "question-to-answer"
LLM reranking	Use the LLM itself to judge candidate relevance	Sees query + document together, catches nuance
PageIndex	Tree-based, vectorless retrieval via LLM reasoning	Preserves document structure, 54% → 98.7% on FinanceBench
Context engineering	Being surgical about every token in the prompt	Every token added is a bet against accuracy

Homework from the Lecture

Challenge 1: Cross-Encoder Reranking — Replace the LLM reranker with `cross-encoder/ms-marco-MiniLM-L-6-v2`. Compare speed (cross-encoder should be much faster) and quality (LLM reranker might be slightly better on nuanced queries).

Challenge 2: Multi-Query Retrieval — Instead of one search, generate 3 different phrasings of the question, search with each, union the results and deduplicate. Compare recall with HyDE — they solve the same problem (query-document gap) from different angles.

Key Diagrams:



Here's the full pipeline. Now the upgrade path showing what each level fixes:

